

Otimizando Pipelines com Tracing de Alta Performance

Python Brasil 2025

<https://github.com/andre-sb/PalestraPerfetto.git>

Apresentação dos Palestrantes

- Isabelle Serique

Desenvolvedora de software pelo Instituto de Pesquisas Eldorado.

Pesquisadora em Ciência de Dados, Otimização e Inteligência Artificial.

Objetivo: inspirar mulheres a ingressar na área tech.

- André Braga

Engenheiro com 30 anos de experiência em sistemas embarcados, otimização, sistemas críticos, Python, certificação e engenharia de software. Consultor do Instituto de Pesquisas Eldorado.

Quem nunca?

- Herdei um projeto grande que está em operação há mais de 10 anos.
- Seria bom entender a dinâmica da execução dos scripts antes que algo de ruim aconteça, né?
- Estourou uma exceção.
O que levou o fluxo da execução até o ponto onde ela estourou?
- Quais eram os valores dessas variáveis nesse ponto?
- Está tudo funcionando bem, mas está muito lento.
O que dá para melhorar sem bagunçar?

A evolução

Mostra alguns valores no terminal

```
print (f"{variavel=}")
```



É, isso ajuda. Vamos deixar definitivo e salvar o histórico?

```
import logging
```



Tenho gigabytes de logs. Conhece alguma ferramenta pra me ajudar a analisar?

Sim. Muitas!

(só um exemplo: <https://profilerpedia.markhansen.co.nz>)

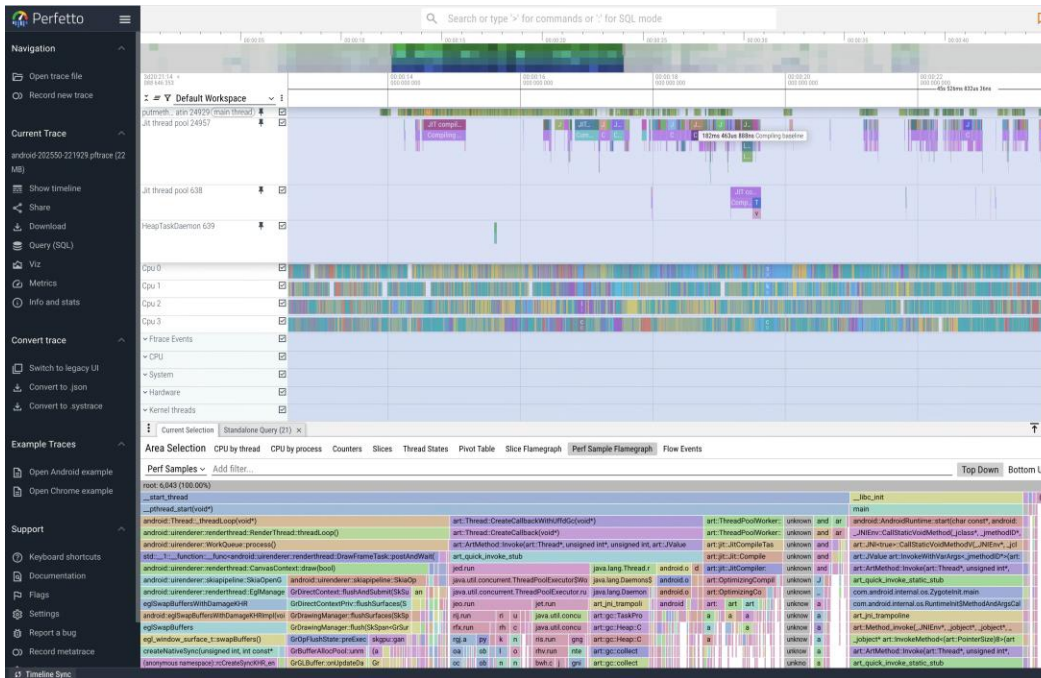


Acho que vou experimentar essa aqui...

Gostei, vou continuar com essa.

Perfetto

- Perfetto é a ferramenta de tracing padrão do Android e do Chrome.
<https://perfetto.dev>
- Integrado nativamente aos principais componentes desses sistemas.
- Interface visual poderosa para análise de eventos.
- Permite [queries SQL](#) para filtrar e investigar dados específicos.



<https://perfetto.dev/docs/getting-started/system-tracing>

- **Tracing (rastreamento)**

Registro detalhado de todo o fluxo de execução.

Contém informação suficiente para explicar o motivo do código ter chegado até um ponto determinado e causado o comportamento observado.

Teoricamente não seria necessário reproduzir um bug, pois está tudo registrado no arquivo de rastreamento.

- **Profiling (perfilamento)**

Registro por amostragem de recursos do programa: pilha de funções, uso de memória, uso da CPU.

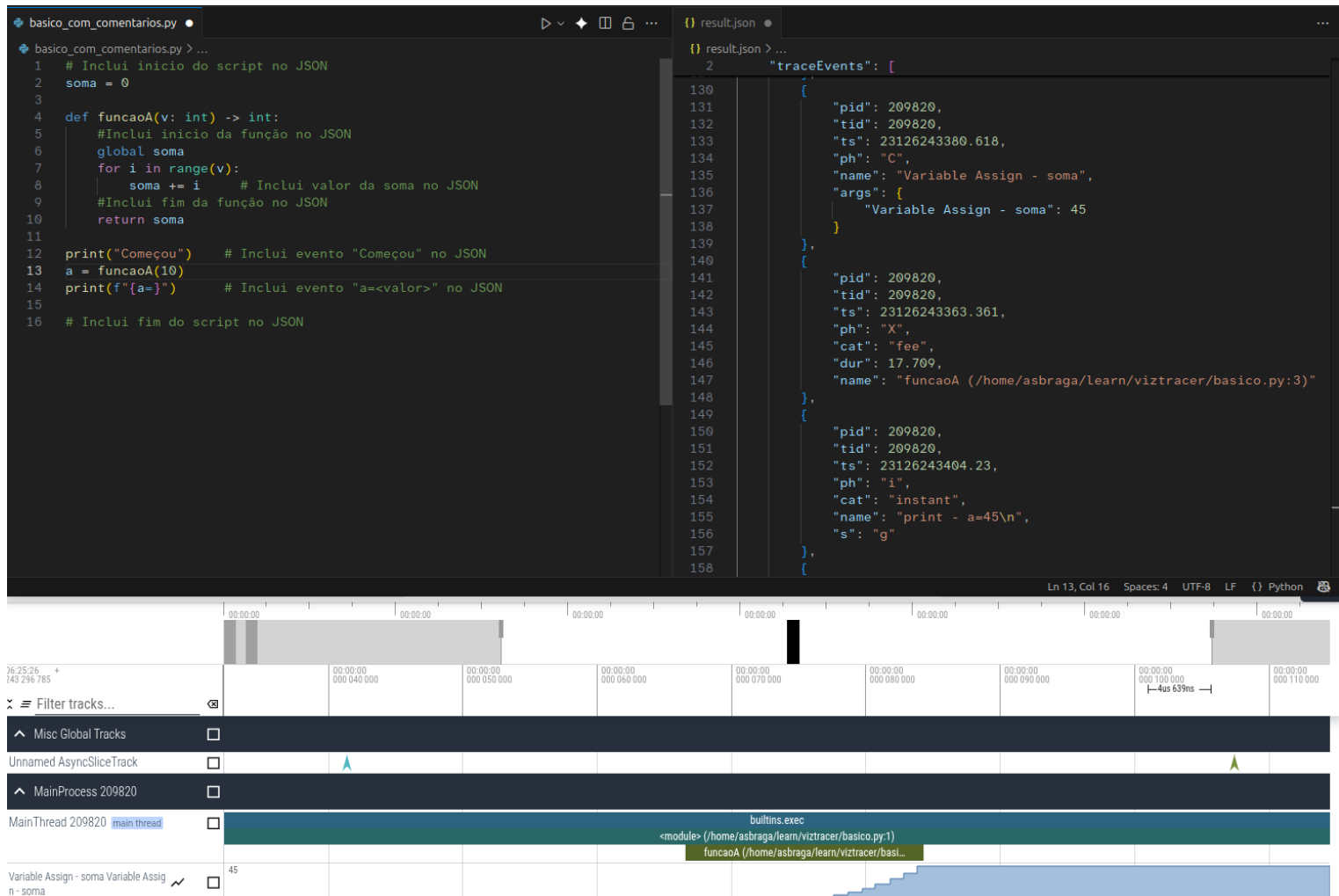
Tem uma natureza mais estatística.

<https://android.googlesource.com/platform/external/perfetto/+refs/heads/main/docs/tracing-101.md>

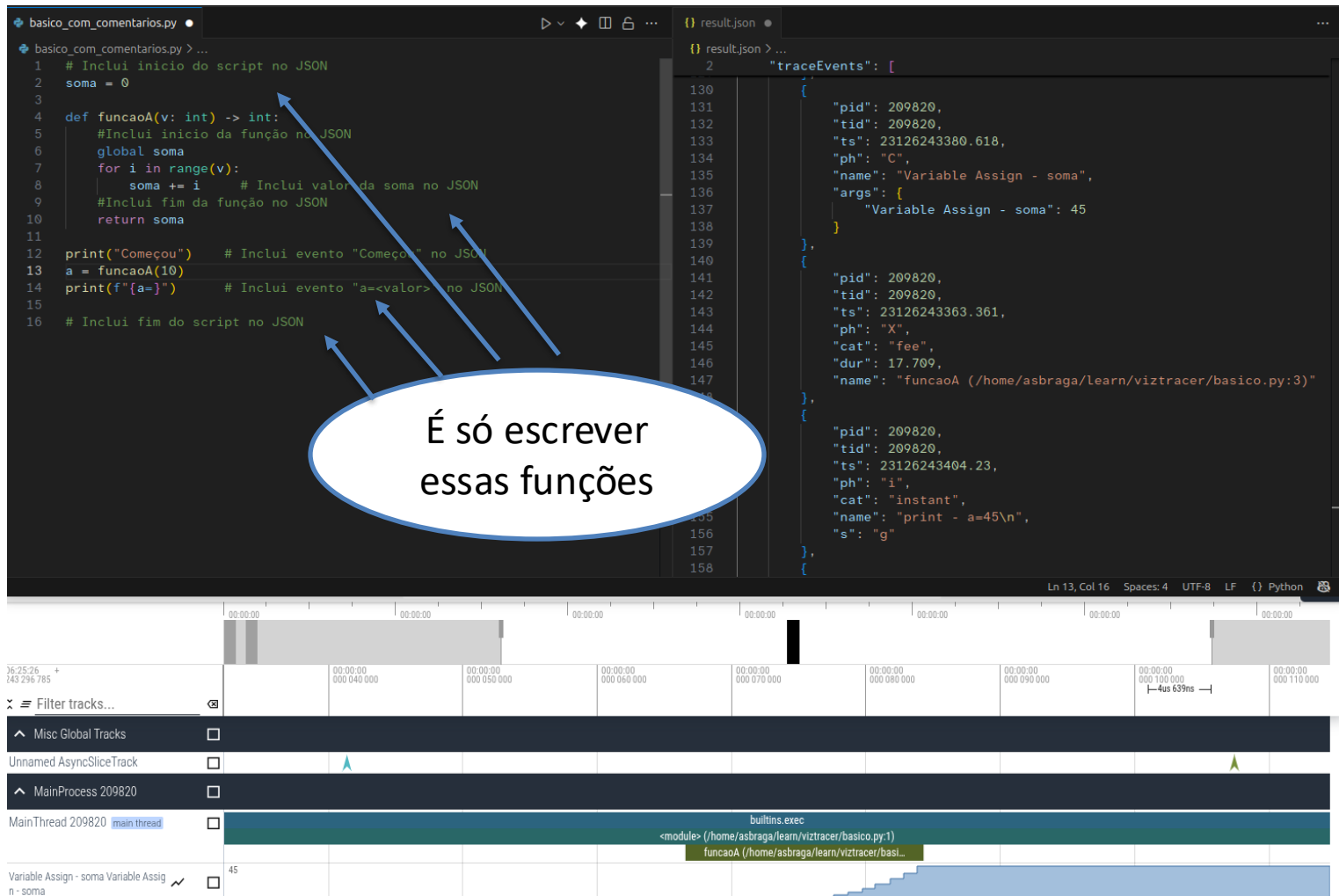
E os meus scripts Python?

- Perfetto é fortemente integrado com Android e Chrome, mas podemos usá-lo para analisar os nossos scripts independente da plataforma em que eles rodam.
- Basta gerar o arquivo de trace no formato JSON do Perfetto.
<https://docs.google.com/document/d/1CvAClvFfyA5R-PhYUmn5OOQtYMH4h6l0nSsKchNAySU>
- Ele aceita outros formatos, mas vamos ficar nesse
- Eventos:
 - **Começo-fim** : mostra a duração de uma execução, ideal para visualizar funções e trechos de código.
 - **Instantâneo**: mostra uma ocorrência de um evento que não tem duração.
 - **Contador**: mostra um gráfico de um valor ao longo do tempo.
 - ...outros...

(demonstração)



(demonstração)

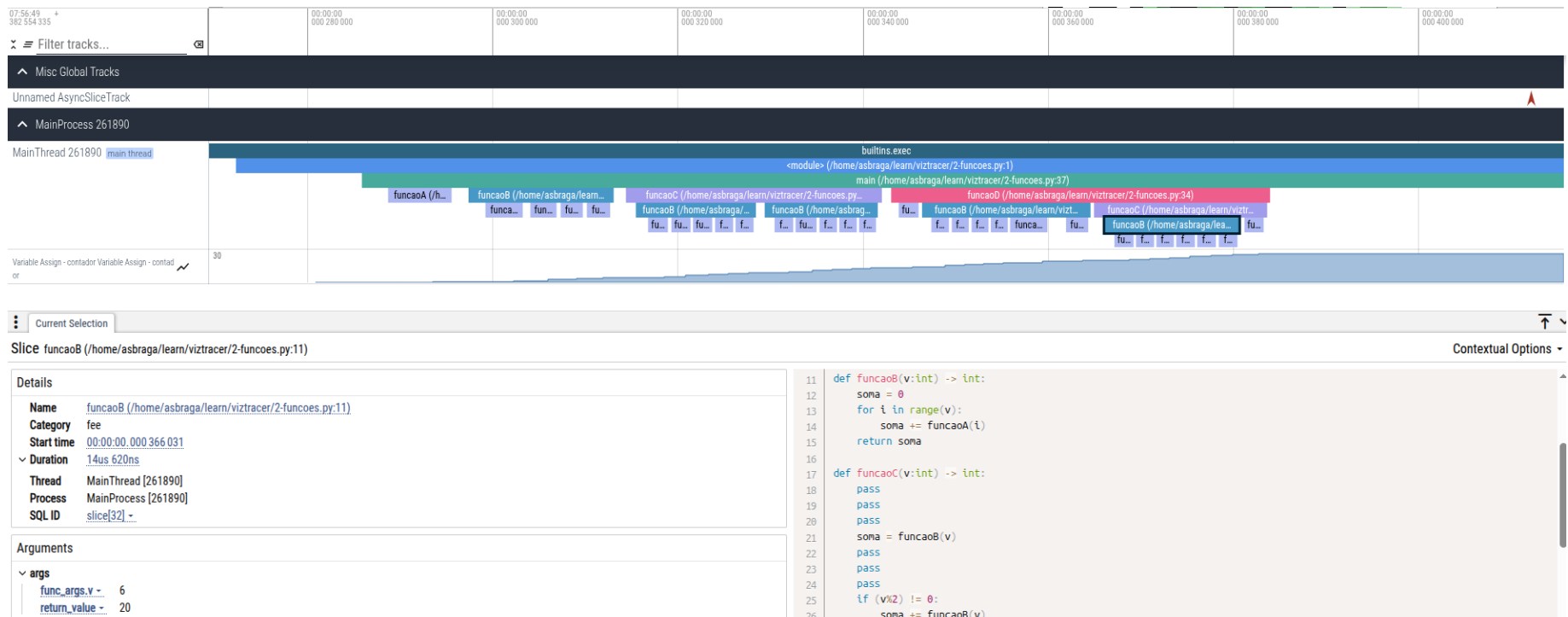


Eis que surge o viztracer

- Ele instrumenta o código para você!
<https://github.com/gaogaotiantian/viztracer>
<https://viztracer.readthedocs.io/en/stable/index.html>
- `viztracer arquivo.py`
- `viztracer --log_number variavel -- arquivo.py`
- `viztracer --log_print -- arquivo.py`
- `viztracer --log_number variavel --log_func_args --log_func_retval --log_print -- arquivo.py`
- E de quebra você ainda ganha um visualizador na máquina local:
`vizviewer result.json`

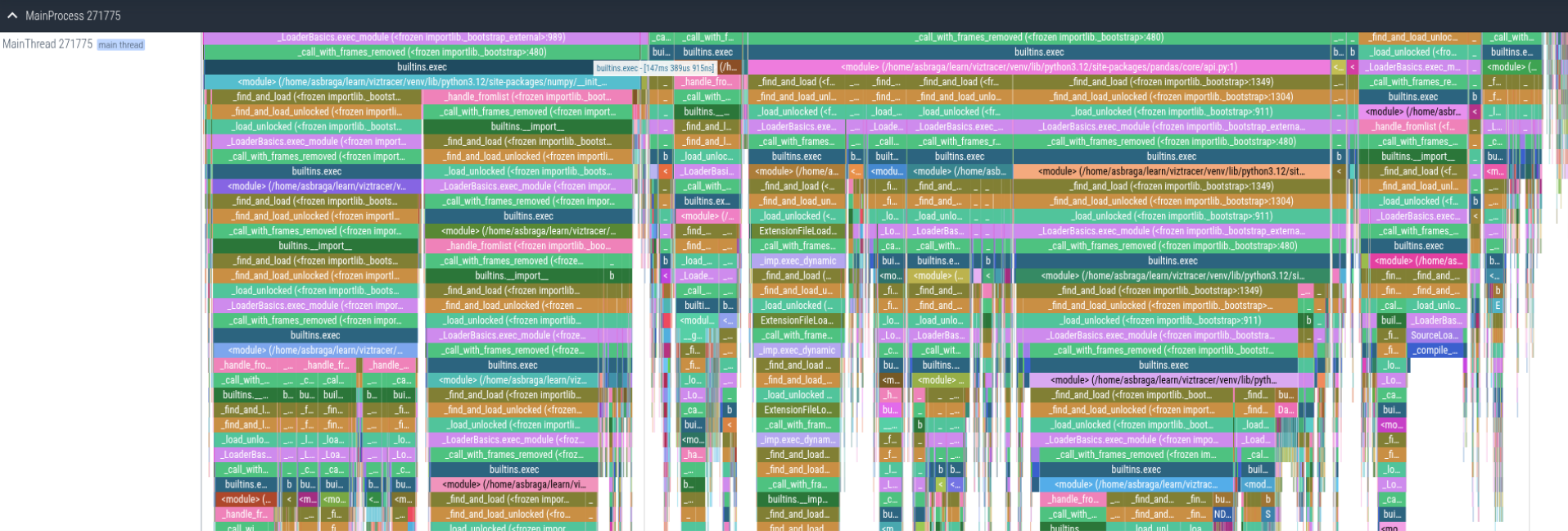
(demonstração)

- `viztracer --log_number contador --log_func_args --log_func_retval --log_print --ignore_frozen -- 2-funcoes.py`



(demonstração)

- `viztracer 3-pandas.py`
`viztracer --max_stack_depth 5 --min_duration 0.1ms 3-pandas.py`
`viztracer --ignore_frozen 3-pandas.py`



Você já sabe, mas não custa lembrar...

- “We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil.” Donald Knuth
- “Rules of Optimization:
Rule 1: Don't do it.
Rule 2 (for experts only): Don't do it yet.” M. A. Jackson
- “More computing sins are committed in the name of efficiency (without necessarily achieving it) than for any other single reason - including blind stupidity.” William A. Wulf
- Não se deixe seduzir pela vontade otimizar ao máximo o seu código!

Você já sabe, mas não custa lembrar...

- Cuidado com regras pré-concebidas, macetes, dicas, construções mágicas que "*são mais performáticas*"

“... empregue o mesmo ceticismo que usamos ao comprar um carro usado...”
Carl Sagan

- Agora é hora de ser científico
 - Estabeleça objetivos de desempenho
 - Faça medições no seu código: Tempo de execução, uso de CPU e memória, ...
- Uma construção complicada que economiza 1 μ s só será significativa se você rodar o código muitos milhões de vezes, mas se um colega não entendê-la, levará semanas para resolver um bug.

Perguntas

- Espaço aberto para dúvidas e discussões.
- Compartilhamento de experiências com tracing.
- Exploração de casos específicos dos participantes.